

Statistical Analysis on Income Classification

Visar Sokoli

December 2024

1 Introduction

Income classification plays a crucial role in understanding economic differences, enabling organizations and government itself to develop targeted policies for resource allocation and robust industrial improvement. Despite the extensive availability of economic data, accurately predicting income continues to be a complex task due to the intricacies and dependencies on hundreds of different social and economic features. The dataset which will be used for the majority of the paper is one from the UC Irvine Machine Learning Repository, and it provides thousands of observations of individuals with a range of different features and classifies them based on a certain income threshold. This paper seeks to leverage different machine learning and statistical techniques to classify income levels; offering insights that could improve predictive accuracy and enhance decision making in economic policy planning. The objective will be to analyze this dataset containing demographic and socioeconomic features, apply various classification models, and evaluate their performances based on different predictive scores discussed later on. The main machine learning algorithms used here will be random forests, K-nearest neighbors, gradient & XG boosting, decision tree, among others. Using these models and different optimization techniques, we strive to identify key features in predicting income and increasing predictive power.

2 Data Description

The dataset contains around forty-eight-thousand observations after dropping rows with empty entries, but for simplicity a random sample of ten-thousand observations were used for simplicity and convenience. Just in case, the analysis was done on the full dataset and the results did not differ from the analysis done in this paper on just the random sample collected. The dataset comprises fourteen features and one target variable being an indicator that tells us if an individual income is above or below fifty-thousand dollars per year. Of the fourteen features, eight were categorical and six were numerical. The features are listed as follows grouped in the two categories with some percentages:

- | | |
|--|---|
| • Workclass - Sector of work (Private, Gov, etc.) | • Age |
| • Education - Level of education completed | • Education num - # of years spent in education |
| • Marital-status - (Married, Single, Divorced, etc.) | • Fnlwgt - Associated Weight |
| • Occupation - Field of Occupation | • Capital gain |
| • Relationship - (Husband, Wife, Single, etc.) | • Capital Loss |
| • Race - (86% White, 9% Black, etc.) | • Hours per week - Hours worked /week |
| • Sex - (67% Male, 33% Female) | |
| • Native-Country - (U.S., other) | |

Next we combined certain entries in the data to reduce multicollinearity and future model complexity when encoding is eventually done. All entry combining was performed carefully using correlation coefficients as a way to conclude if features were correlated enough to be combined. First, all entries that were grades in high school were combined into the entry ‘HS-no_grad’. Similarly, all entries that were grades in grade school were combined into the entry ‘Grade_school’. Secondly, entries in workclass that signified government jobs were combined to ‘Gov_jobs’ along with entries implying self-employment were combined also. Finally, in the Native-country feature, only the most frequently appearing counties of the United States and Mexico were kept, and the rest were replaced with ‘Other-country’ along with some other small logical changes. To better understand the datasets structure and demographic distribution, we can look at how education level is distributed

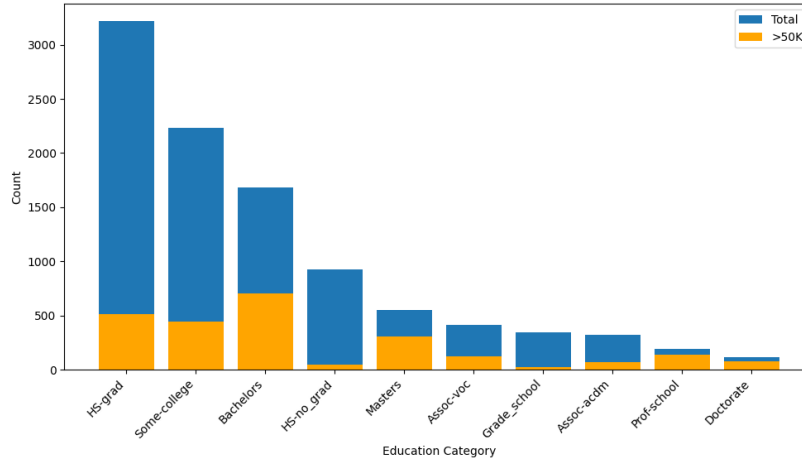
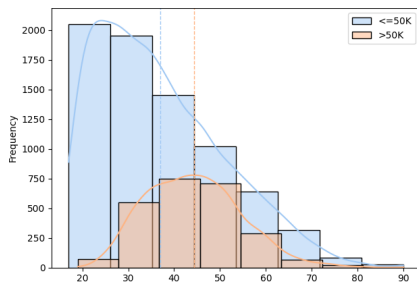
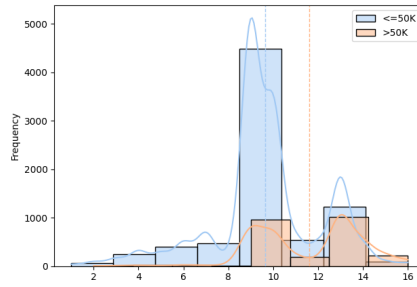


Figure 1: Education Category Dist.

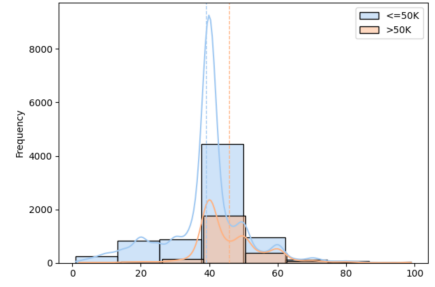
It is clear to see from the figure above before any analysis that the level of education completed will be a big factor in predicting income because there is a greater proportion of individuals who make more in the levels of education that take longer to complete. Now we can take a look at how some of the numerical features are distributed and if they might have any clear association with income. The three obvious features are age, education number, and hours worked per week.



(a) Dist. of Age by Income



(b) Dist. of Education by Income



(c) Dist. of hrs/week by Income

From the histograms we can tell that individuals who made more money were on average older, spent more time in school, and work more hours per week than individuals who made less money. Another way of exploring which features had some kind of impact of income is to display the correlation table of income and the numerical features.

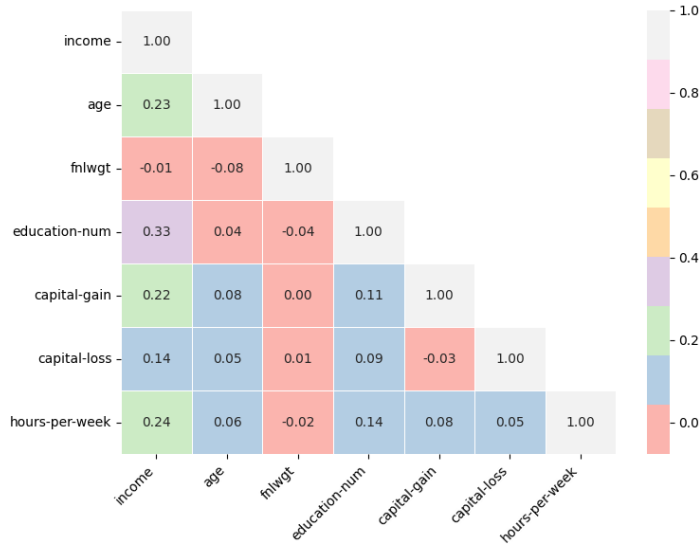


Figure 3: Correlation of Income and Numerical Features

From the correlation table it is obvious that features like age, education number, capital gain/loss, and hours worked per week have an effect on the income an individual earns while fnlwgt doesn't have a noticeable effect on income. Another relationship we see from the correlation table is that the numerical features are not very correlated with one another which prevents multicollinearity for our upcoming analysis. Finally, the eight categorical variables are encoded into the data, increasing our number of features from 14 $\rightarrow$ 58. This encoding basically takes the entries of each categorical feature and turns each of those entries into their own features in the dataset using the numbers $\{0, 1\}$ instead of words. This ensures there is no data leakage (data lost) and our models will be able to handle the relationships between these features and the target variable numerically. This does have a drawback in that our models will take longer with computations due to the significant increase in features, but this negative is outweighed by the new information the models now receive to make better predictions in the long run. Now that the dataset is cleaned and processed we can look at the features that have the highest and lowest correlations, meaning which features tend to increase and decrease income respectively.

Table 1: Correlations between Features and Income

Feature (+)	Correlation	Feature (-)	Correlation
Married-civ-spouse	0.44	Never-married	-0.31
Husband	0.40	Female	-0.22
Education number	0.33	Own-child	-0.22
hours-per-week	0.24	Not-in-family	-0.19

Displayed in the table above are the four features with the highest and lowest correlation with income. Interestingly enough marriage seems to be a big indicator of income overall since married-civ-spouse seems to be the biggest positive indicator of income while never-married is the most negative predictor of income. This is of course due to many reasons, but the main one would be people with higher incomes are in a better position overall to get married and settle down to start a family.

3 Model Development & Optimization

3.1 Introduction & Goals

Before the data is ready to train our different machine learning models, the dataset is first scaled by centering each feature around 0 with a standard deviation of 1. This prevents bias by reducing the scale of our larger features while also improving the model performance of distance based models like KNN and SVR. Overall, scaling the data enhances the performance and stability of many machine learning algorithms we will use. The data is then split randomly into two sets, the training set and testing set, this is done using a conventional 80% training and 20% test. To predict the target variable income, we will be implying a diverse set of machine learning models including K-Nearest Neighbors (KNN), Random Forest, Gradient Boosting, XGBoosting, Decision Tree, alongside a customized neural network, each offering unique and non-linear approaches to handle the complex relationships and patterns in the data. Each model will be trained on the training data, then we will optimize certain parameters to increase performance, followed by each optimized model being evaluated on the testing data. All optimization will be done using cross-validation and a grid search to ensure our models generalize well to unseen data while reducing bias and variance in performance evaluation. The three performance metrics we will be using to evaluate our data will be accuracy, precision, and ROC-AUC score.

1. Accuracy - Measures the overall correctness of the model by calculating the ratio of correctly predicted observations to the total observations.
2. Precision - Measures the correctness of positive predictions by calculating the ratio of true positives to all predicative positives.
3. ROC AUC Score - Evaluates the model's ability to distinguish between classes by measuring the area under the Receiving Operating Characteristic (ROC) curve. It provides insight into a models performance across difference thresholds especially useful for imbalanced datasets.

Using these three metrics ensures a comprehensive evaluation of a model's performance because together they capture different aspects of the its behavior. Finally, each model will be predicting a value of 0 or 1 for every observation in the testing data.

0 $\rightarrow$ False (<50k)

1 $\rightarrow$ True (>50k)

3.2 K-Nearest-Neighbors (KNN)

K-Nearest Neighbors (KNN) is an effective supervised machine learning algorithm commonly used for classification tasks. It operates on the principle of similarity, predicting the class of a data point, in this case above or below 50k, based on the classes of its closest neighbors in the feature space. To classify a new instance, KNN calculates the distances between the instance and all training samples, identifies the K nearest data points, and assigns the majority class among these neighbors. The choice of K significantly affects the model's performance, with smaller values emphasizing local patterns and larger values capturing broader trends. KNN is non-parametric, meaning it makes no assumptions about the data's distribution, making it versatile for a wide range of applications. However, it can be computationally heavy when working with large datasets due to numerous distance calculations.

To begin optimizing KNN we first optimize its main parameter, K neighbors, this is done by performing a 5-fold cross validation on the training set and choosing the K value that optimizes the average cross validation accuracy. The K parameter determines the number of nearest neighbors considered when classifying an observation, with the majority class of the neighbors deciding the classification.

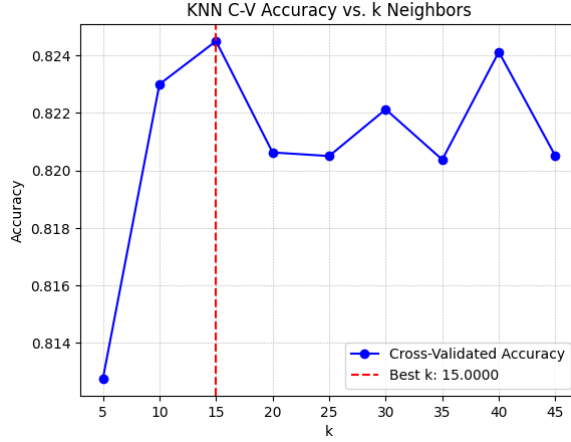


Figure 4: KNN: Optimized K Neighbors

From the figure above the best K value to use is 15 so we operate at this level for the next step in the optimization. In this final step we set up a grid search for five additional parameters of KNN and run a 3-fold cross validation with the K neighbors parameter fixed at 15. This method ensures a comprehensive search for the remaining parameters weights, p, metric, leaf_size, and algorithm. With this optimized KNN model we can finally test it on the test data and see how it performs based on the three metrics we want to evaluate. Additionally, we can see how it performed compared to a standard KNN model with no fine-tuning.

Table 2: KNN Performance

Model	Accuracy	Precision	ROC AUC
Base KNN	0.816	0.630	0.83
Opt. KNN	0.824	0.662	0.86

Table 3: KNN Confusion Matrix

	Predicted <50k	Predicted >50k
Actual <50k	1396	128
Actual >50k	225	251

The optimized KNN model was able to perform decently well achieving 82.4% accuracy along with a good ROC AUC score making it a strong option, but maybe outshone by more complex models used later in the paper. In conclusion, based on the tables above the optimized KNN model demonstrated slight improvements in all metrics compared to the base model, highlighting its ability to refine prediction ability with the help of fine-tuning. We hope to achieve further improvements with some of the other more complex models.

3.3 Random Forests

Random Forest classification is a powerful and widely used ensemble learning technique that combines multiple decision trees to make more accurate and stable predictions. In a Random Forest model, each tree is built by randomly selecting subsets of the features and training data, which reduces overfitting and improves the model's ability to generalize to new data. The final prediction is made by aggregating the individual predictions from each tree, typically using majority voting for

classification tasks. The algorithm excels in classification especially when the relationships between features and the target variable is non-linear. This method is also valuable for feature importance analysis, as it can assess the contribution of each feature to the model’s predictive power.

To begin optimizing the random forests classifier the same method will be used as before, that it first optimizing the main parameter and then using other methods to obtain a combination of values for the best parameters. The main parameter in RFC is n estimators, so we use a 3-fold cross validation on the training set and identify the value of n that optimizes the average cross validation accuracy. The n estimators parameter determines the number of decision trees to build for the classification task.

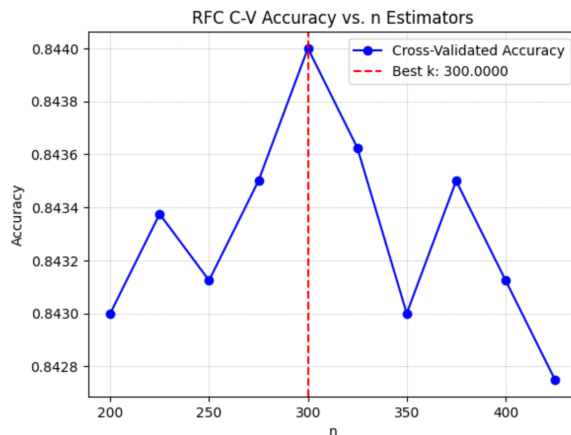


Figure 5: RFC: Optimized n Estimators

From the figure above the optimal n value to use is 300 so we operate at this level for the next step in the optimization. In the final step we set up a grid search for three additional parameters of RFC and run a 3-fold cross validation with the n estimators parameter fixed at 300. The three other parameters optimized are also significant in possibly increasing performance metrics because they essentially control different constraints on the multiple trees RFC builds. With this optimized RFC model we can finally test it on the test data and see how it performs based on the three metrics we want to evaluate. Additionally, we can see how it performed compared to a standard random forest model with no fine-tuning.

Table 4: RFC Performance

Model	Accuracy	Precision	ROC AUC
Base RFC	0.846	0.706	0.90
Opt. RFC	0.860	0.770	0.91

Table 5: RFC Confusion Matrix

	Predicted <50k	Predicted >50k
Actual <50k	1445	79
Actual >50k	201	275

The optimized RFC model was able to perform decently well achieving 86% accuracy along with a strong ROC AUC score making it a strong option. Another positive from this model is that the optimized model was able to outperform the base model by a good margin compared to the

KNN model used. This shows that RFC benefits greatly from fine-tuning parameters and can even improve performance further by using a higher fold cross validation for fine tuning more even more parameters. Another important feature of RFC, is that it can calculate the features with the largest importance, essentially identifying the features with the biggest impact on predicting income.

Table 6: RFC Feature Importance

Feature	Importance
Capital-gain	0.141
Married-civ-spouse	0.120
education-num	0.112
Husband	0.088
age	0.085

The table above shows that capital gain was the best indicator in predicting income and this is logical because individuals who make more money are able to invest extra money towards different vehicles in investment returning a capital gain. The other features that appear in the table also appeared in our preliminary graphs before any analysis was done.

3.4 XGBoost

XGBoost (Extreme Gradient Boosting) is a machine learning algorithm that builds multiple decision trees to improve metrics like accuracy. It sequentially adds trees, each focused on correcting the errors of the previous ones, by minimizing a specific loss function. XGBoost is an efficient machine learning model built for speed and performance, with added features like regularization to prevent overfitting. Its final predictions are made by combining the outputs of all trees, typically through weighted averages.

To begin optimizing the XGBoosting classifier a slightly different approach will be used. XGBoost has a large amount of parameters available to optimize and has two parameters in specific that can drastically effect performance. To complement this fact, the two main parameters were optimized one after the other in sequence. The two parameters are Max depth and n estimators, and they both effect the models complexity and decision making greatly. The max depth parameter determines the maximum depth of each tree and the n estimators parameter specifies the number of trees to train for classification. First, the max depth parameter was optimized by identifying the value it achieves the highest cross validation accuracy. Following, this the n estimators value was optimized the same way using a 3-fold cross validation for both parameters.

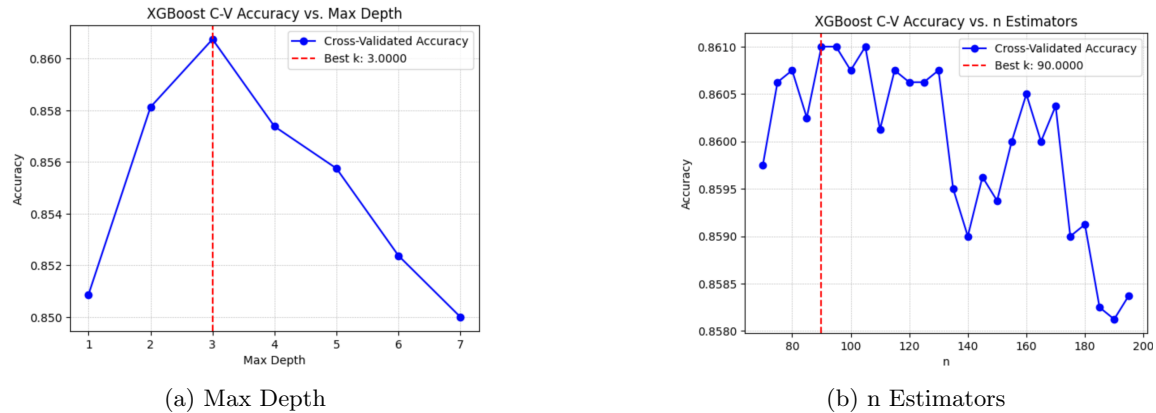


Figure 6: XBG Parameter Optimization

From the figures above the optimal max depth value to use is 3 so we operate at this level when optimizing the n estimators value which turned out to be 90. In the final step we set up a grid search for eight additional parameters of XGB and run a 5-fold cross validation with the two already optimized parameters fixed. XGB is a model with a large quantity of parameters and it was tempting to see how far this process can be pushed while checking for overfitting using learning curves. With this comprehensively optimized XGBoost model we can finally test it on the test data and see how it performs based on the three metrics we want to evaluate.

Table 7: XGBoost Performance

Model	Accuracy	Precision	ROC AUC
Base XGB	0.863	0.751	0.92
Opt. XGB	0.868	0.780	0.92

Table 8: XGBoost Confusion Matrix

	Predicted <50k	Predicted >50k
Actual <50k	1441	83
Actual >50k	182	294

The optimized XGBoost model was able to perform the best out of any model tested so far, achieving almost 87% accuracy along with a fantastic ROC AUC score making it the strongest model in predicting income. Interestingly enough although XGBoost had the most parameter fine-tuning it barely outperformed its base model indicating that the base model was already capable of capturing the relationship of the features and income without the need of parameter tuning. XGB can also be used to explore what kind of effect the most important features had on income. Instead of just listing the features based on importance, a SHAP graph can be used to differentiate which features have positive or negative effect on income.



Figure 7: XGBoost: SHAP Plot

The figure above shows how the top eight features, in terms of importance, effect the level of income in the dataset. The red points are higher feature level while blue points indicate lower feature levels. For example the feature with the largest positive impact was number of years spent in education because it has red points (individuals who spent a longer time in education) towards the larger positive values meaning they had a larger income on average. Based on the plot individuals

who had higher values in age, number of years in education, capital gain, and hours worked per week on average were more likely to make more than fifty-thousand dollars. On the other hand, individuals who were never married or worked in the other service category more than likely made under this threshold.

3.5 Decision Trees

Decision tree classification is a method that organizes data into a tree-like structure to make predictions based on input features. The tree is composed of nodes representing features, branches representing decision rules, and leaves indicating class labels. At each step, the algorithm splits the data into subsets based on a feature and a correspondent threshold. Similar to optimizing the parameters of other machine learning models, a decision tree classifier model is pruned which means the nodes with the least predictive power are removed which both increases overall accuracy and prevent overfitting which decision trees are prone to. The parameter ccp alpha (cost-complexity pruning alpha) controls the trade-off between tree complexity and training accuracy. A higher ccp alpha prunes the tree more aggressively. A 5-fold cross validation along with a grid search was used to sort through the possible ccp alpha values and arrive at the value that maximized average cross validation accuracy. This methods allows fore a model that generalizes well to new unseen data and systemic validation ensures a robust model.

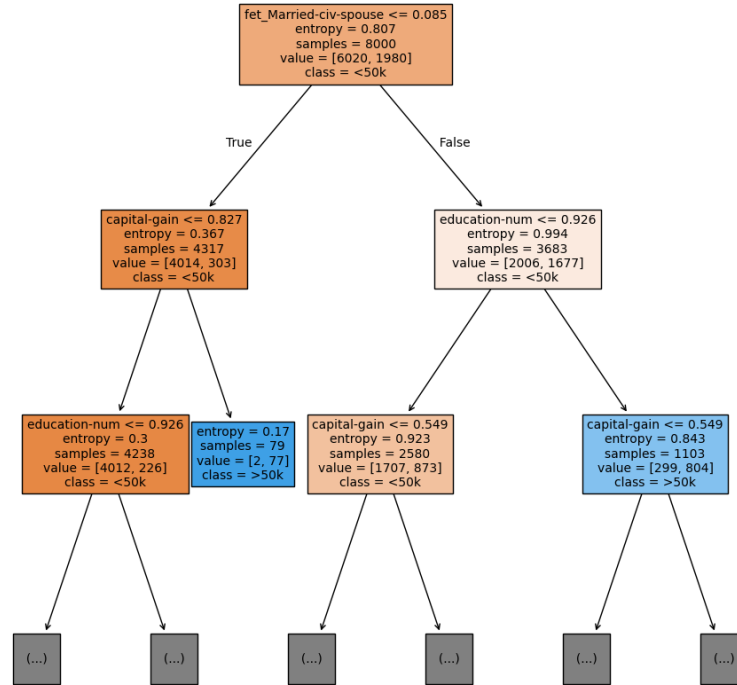


Figure 8: Pruned Decision Tree (First few Braches)

The figure above shows the decision tree and how it classifies observations based on a feature and threshold then splits the tree from their diving the training set into smaller and smaller subsets. The very first node tells us that if the individual is married than they move down to the left and if not than they move down to the right. The node down to the left, splits that smaller subset of individuals based on whether or not their capital gain is above a certain level. As seen if individuals

have a capital gain over that threshold they move down to the right into a dark blue terminal node concluding that those individuals more than likely make over fifty thousand dollars. If you then check that dark blue terminal node it contains seventy-nine individuals and all but two of them make over that income threshold. So our pruned decision at a first glance seems to be making good classifications, but lets take a look at its performance metrics on the test set.

Table 9: Decision Tree Performance

Model	Accuracy	Precision	ROC AUC
Base Tree	0.801	0.578	0.73
Pruned Tree	0.846	0.711	0.89

Table 10: Decision Tree Confusion Matrix

	Predicted <50k	Predicted >50k
Actual <50k	1410	114
Actual >50k	195	281

Table 11: Decision Tree Confusion Matrix

The pruned decision tree was able to perform at almost a 85% accuracy level while maintaining a good ROC AUC score making it a strong option, but not necessarily the best out of the models tested so far. The interesting observation of the scores, is how pruning the decision tree increased all three performance metric by quite the margin. It shows how powerful simply pruning a decision tree can be to increasing predicative power while at the same time preventing overfitting.

3.6 Neural Network

As an additional experiment, a custom neural network was designed and implemented to fit the data, providing results from a model built from scratch, in contrast to the pre-built models used previously. Neural networks are powerful machine learning algorithms that consist of layers of interconnected nodes, or neurons, that process input data and extract patterns to make predictions. Information flows through these layers, starting with the input layer, which receives raw data, followed by hidden layers that transform the data using mathematical operations, and ending with an output layer that produces predictions. The network learns by adjusting the connections between neurons during training, using optimization techniques to minimize error.

The neural networks' parameters were also optimize in the goal to increase performance. A 5-fold cross validation along with a grid search of multiple parameters key to the models performance were used to optimize the network. Some of the main parameters optimized were the number of layers, number of neurons, and activation. The number of layers specifies the hidden layers in the network, which when increased allow the model to learn more hierarchical features, but come at the risk of high compilation time and overfitting. The number of neurons control the models complexity per layer and the activation controls which activation function it uses. Both the base and optimized neural network were trained and evaluated on the test data and produced the following results and metrics.

The optimized neural network was able to match the performances of some of the previous models used in the paper, with an accuracy just above 85% and a solid ROC AUC score making it a strong option for predicting income. Another positive of the optimized neural network is that it was able to out perform the base network in all metrics, but especially in the ROC AUC score which is the most well rounded metric to look at in terms of performance.

Table 12: Neural Network Performance

Model	Accuracy	Precision	ROC AUC
Base Network	0.835	0.654	0.75
Opt. Network	0.844	0.734	0.89

Table 13: Neural Network Confusion Matrix

	Predicted <50k	Predicted >50k
Actual <50k	1431	93
Actual >50k	219	257

4 Feature Reduction

4.1 Introduction & Goals

Before moving to the conclusion section where we will compare all of the machine learning models that were used in classifying income, a type of feature selection can be applied before fitting any models in the hopes of reducing the number of features in our dataset which reduce the complexity and computation time of the models while losing as little information as possible. Principal Component Analysis (PCA) is a dimensionality reduction technique used to transform high-dimensional data into a lower-dimensional space while preserving as much variance as possible. It identifies the directions (principal components) along which the variation in the data is maximized. In classification problems, PCA is useful because it reduces the number of input features, which can help improve model performance by eliminating noise, reducing overfitting, and lowering computational complexity. This is especially beneficial when the original dataset has many correlated or redundant features like our own. By focusing on the most important components, PCA simplifies the data structure while retaining the information that contributes most to the predictive power.

The goal of this section will be to explore if applying PCA, prior to any training of models, will improve model performance and to seek which features were the biggest predictors in the principal components. All of the same models and optimization techniques will be used, but with obviously different parameter values since we are incorporating a method of feature reduction in the case.

4.2 Principal Component Analysis

The first step to using PCA is to apply it to our training data and observing how many components we can drop from our dataset while retaining as much variance as possible. Reminder that our original dataset after encoding has 58 features, we can apply PCA to see how many features are actually needed to retain a high level of variance using a simple plot.

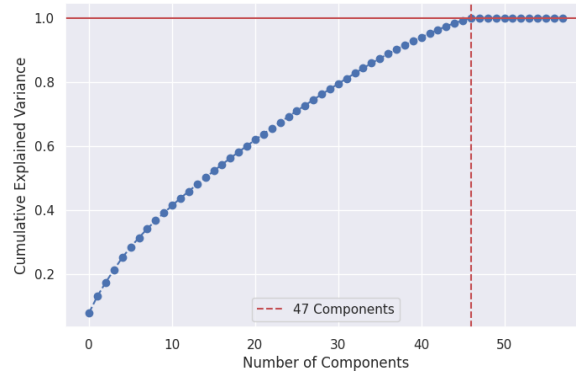


Figure 9: Explained Variance by Number of Components

From the figure plot above we can immediately see that the points are forming a concave shape. What this means is that the more components we decide to use the less they add to the overall explained variance. This is called diminishing returns so we should obviously cutoff the number of components somewhere this return is minuscule as to lose the least explained variance, a way of doing this is utilizing the elbow method. As seen in the figure, we place that cutoff at 47 features because that is where the points flatten out almost completely meaning those last 11 features add almost nothing to the explained variance. So PCA was able to reduce our number of features from $58 \rightarrow 47$ which will in turn lower complexity and decrease computation time for the models.

Now we can explore the principal components that were created using various plots. First we can take a look at how much of the explained variance the first couple of principal components contribute to.

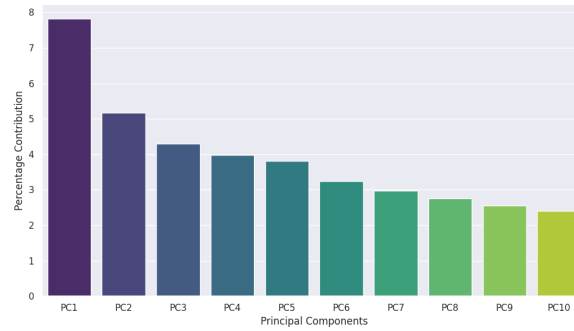


Figure 10: Overall Contribution of First 10 PCA Features

From the figure above we can observe that a great chunk of the variance is explained by the first couple principal components. Specifically, from the figure we observe that the first two principal components explain around 13% of the variance. Another way to explore the influence the different features have on income is through the through analyzing which features are used to create each principal component, specifically the components that comprise most of the explained variance. We can take the most important features that comprise the first two principal components and list them in terms on importance and what kind of influence they typically have on income in our dataset.

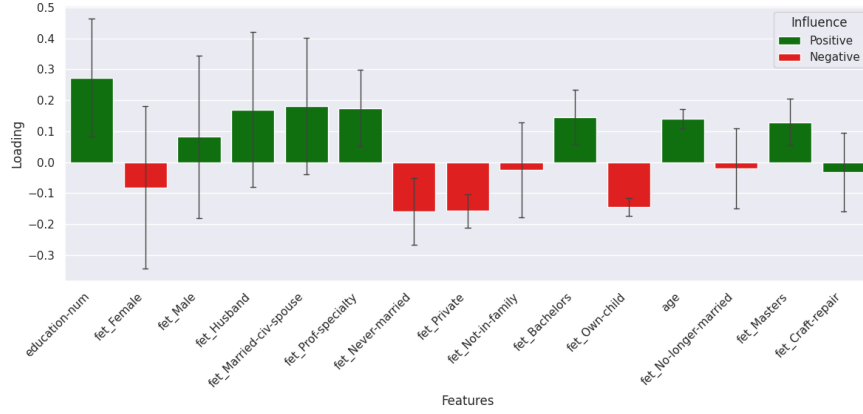


Figure 11: Influence of Most Important Features in PC1 & PC2

From this figure we can observe some of the features previously discussed that influenced income in positive ways like years in education, Husbands, and individuals that are married. However, the figure also provides the relative scale of influence of these features, but also lists features some of the variables that had a negative influence on income level like females, individuals who are not married, and people who work in the private industry. This shows some interesting insights to income, like females are actually a negative indicator to income status, which can be associated to many different economic and political reasons. Another thing we see is that working in the private industry has a negative influence on income which may seem unintuitive at first, but given a further look into the dataset we see that many private professions in the dataset are manual labor jobs which antisocially pay less than other profession. This provides the insight that our dataset may be skewed for the reason of possibly over representing these populations relative to their actual proportion in society. Finally, another advantage of utilizing PCA for feature space reduction is we can see how our target variable, income level, is distributed relative to the values of the different principal components. For example, we can observe which principal components have positive and negative influence on income level helping us better understand their significance.

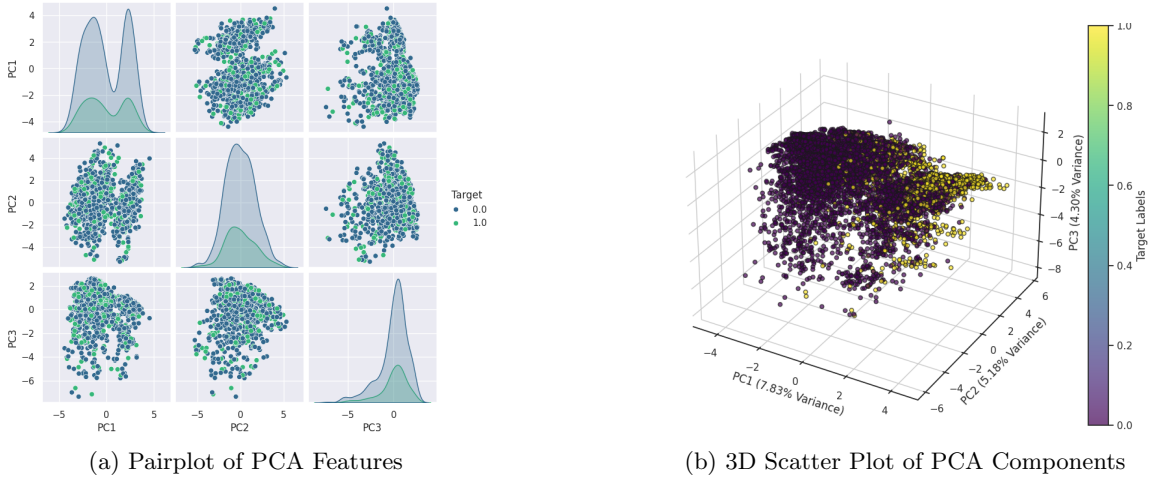


Figure 12: PCA Income Distribution

From the scatter plots above we are able to observe the distribution of income classes relative to their different PCA component values. This is crucial to the understanding of how the most

important features are collected and form different subsets to help better classify individuals in different situations to our two income classes. Our next and final step will be to utilize this new dataset with reduced features via PCA to determine how the same models we utilized before behave and provide results given these transformations.

4.3 Model Results

Using PCA we obtained new training and test subsets of the data that contain 47 principal components instead of our usual 58 features we used to train and test our different machine learning models previously. We then trained the some of the same models we utilized in the lasts section using our new training set with components. All models parameters were then optimized using the same methods we discussed in depth in the previous section such as cross-validation, grid search, and pruning. With this we are able to compare the performance metrics of these models with the new use of PCA and observe how they differ to our previous results.

Table 14: Model Performance using PCA

Model	Accuracy	Precision	ROC AUC
Base KNN	0.82	0.63	0.83
Opt. KNN	0.83	0.69	0.88
Base RFC	0.83	0.69	0.87
Opt. RFC	0.83	0.74	0.88
Base Tree	0.80	0.58	0.72
Pruned Tree	0.81	0.71	0.84

From the performance table we clearly observe that our models under PCA don't perform to the level of the models of the previous section utilizing all features. Using PCA can sometimes reduce accuracy, precision, and ROC AUC because it transforms the original features into principal components that may not retain all the relevant information for predicting the target variable. While PCA minimizes variance loss, it does not specifically focus on preserving the relationships between features and the target, potentially discarding important predictive information. However, even though our model performances aren't at the same level, we do still observe that optimizing our models still provide good results in increasing different metrics drastically at times.

5 Conclusion

In conclusion, we looked at how different machine learning models were able to perform in the task of clarifying income level given a variety of different social, economic, and geographical features of different individuals in the dataset. In this paper we additionally utilized models like random forests and xgboost to provide a deeper understanding of feature importance and impact on income level. We also made use of principal component analysis to explore the different intricacies of the features and how important variable were utilized to create components to help us better understand there relation with income classification. Lastly, every model will be ranked in terms of performance to see which model is the strongest option in classifying income. For additional information on he models, both the true positive rate (precision) and true negative rate (specificity) will be included.

Table 15: Optimized Model Performances

Model	Accuracy	ROC AUC	Precision	Specificity
XGBoost	86.8%	0.92	78.0%	88.8%
Random Forest	86.0%	0.91	77.0%	87.8%
Decision Tree	84.6%	0.89	71.1%	87.9%
Neural Network	84.4%	0.89	73.4%	86.7%
KNN	82.4%	0.86	66.2%	86.1%

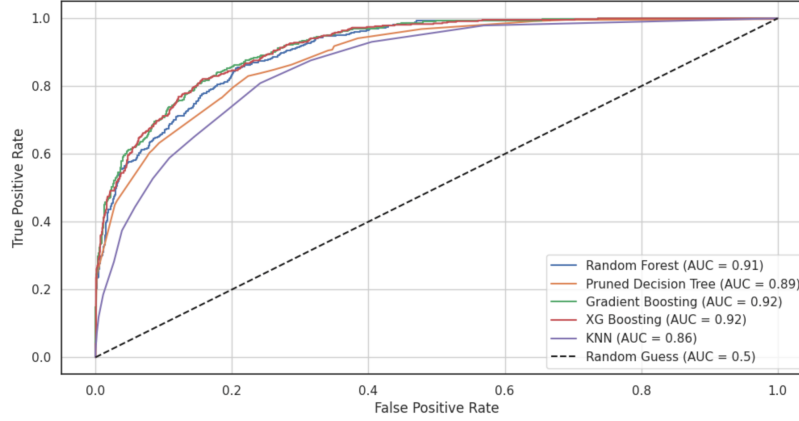


Figure 13: ROC Curves for Different Models

All the models worked with in this paper had somewhat similar results, which shows that each model was able to fairly capture the complex relationships between features and income. From the table above, XGBoost outperforms all other optimized models in all four metrics we tested meaning it is the best candidate we have for classifying income based on the features available to us. The optimized XGBoost model performing the best doesn't come to a surprise because it is one of the most commonly used boosting models in machine learning and known for its accuracy and efficiency. Another reason for XGBoost to slightly outperform the other models could be due to more of its parameters being optimized than other models which could effect its performance. Some ways of improving model performance in the paper could be to apply different feature selection methods, use different models, and optimizing nearly all parameters in each model using an even more extensive grid search and cross validation. Overall, these results in classifying income could help different governments and corporations understand complex relationships with income, inequalities in certain fields, and achieve a more equitable distribution of capital.

Appendix

Dataset

The dataset used in this analysis is the **Adult Dataset**, available at the UC Irvine online repository: <https://archive.ics.uci.edu/dataset/2/adult>

Implementation

All coding and plots were performed using **Python**. The following key libraries and functions were used frequently throughout the analysis:

- **pandas** – For data manipulation and preprocessing.
- **numpy** – For numerical operations.
- **matplotlib** and **seaborn** – For data visualization and plotting.
- **sklearn** – For model training, evaluation, preprocessing tools, grid search, and cross validation.
- **xgboost** – For implementing and optimizing the XGBoost classification model.
- **shap** – For feature importance visualization and analysis.
- **tenserflow & keras** – For implementing and optimizing customized neural network.

Optimized Model Parameters

The optimized hyperparameters for the classification models used in this study are as follows:

- **K-Nearest-Neighbors:**
 - `n_neighbors = 15`
 - `weights = 'uniform'`
 - `metric = 'manhattan'`
 - `leaf_size = 17`
 - `algorithm = 'ball_tree'`
 - `p = 2`
- **Random Forest**
 - `n_neighbors = 300`
 - `min_samples_split = 20`
 - `min_samples_leaf = 4`
 - `max_depth = 25`
- **XGBoost:**
 - `n_estimators = 90`
 - `max_depth = 3`
 - `learning_rate = 0.3`
 - `subsample = 1`

- `colsample_bytree = 0.4`
- `reg_lambda = 0.05`
- `reg_alpha = 0.1`
- `min_child_weight = 3`
- `gamma = 0.005`

- **Neural Network:**

- `model__optimizer = 'adam'`
- `model__num_neurons = 64`
- `model__num_layers = 2`
- `model__l2_reg = 0.01`
- `model__dropout_rate = 0.0`
- `model__activation = 'relu'`
- `epochs = 20`
- `batch_size = 64`